ONNX·TensorRT Day 2

ONNX 속 들여다보기 — 코드로 읽고, 오류를 고치기

HYUNDAI AI Insight Campus · 온디바이스 AI · ONNX·TensorRT 2/5

2026. 11. 5 (목) 09:00–18:00 · 8교시 · Google Colab (하드웨어 없음)

graph | describe_onnx | opset | dynamic_axes | TracerWarning | onnxslim

강사 : 김정인 · 김구수

오늘의 학습 목표

어제의 4 단계 중 '열어보기' 를 코드로 깊게 — 그리고 어제 막혔던 것을 푼다

1 **파일 구조 읽기**

ModelProto → graph 의 input · node · initializer · output 을 코드로 꺼낸다

2 **describe_onnx**

이름으로 화살표를 따라가고, 가중치 · 속성을 꺼내 그래프 요약표를 만든다

3 **opset · checker**

opset 판 번호를 정하고, checker · shape_inference 로 파일을 점검한다

4 **dynamic_axes**

배치 축을 열어 100 장을 한 번에 — dynamic 은 약속일 뿐임을 안다

5 **조용한 오류 3 가지**

파이썬 if · NumPy 값 · 숫자로 고정한 배치를 찾아 고친다

6 **미지원 연산 · 단순화**

aten:: 오류를 읽고 바꾸며, onnxslim 으로 노드를 줄인다 (11 → 4)

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 매 교시 보기 → 따라하기 → 바꿔보기

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 1 복습 퀴즈 · 4 단계 다시 · 오늘의 지도	퀴즈 + 실습	D2-1 4 단계 다시
2	10:00–10:50	ONNX 파일 = 그래프 — input · node · initializer · output	강의 + 실습	D2-2 파일의 속 구조
3	11:00–11:50	가중치 · 속성 꺼내기 — describe_onnx()	강의 + 실습	D2-3 가중치 · 속성
4	13:00–13:50	opset · checker · shape_inference	강의 + 실습	D2-4 opset 사전 · checker
5	14:00–14:50	배치 크기 자유롭게 — dynamic_axes	강의 + 실습	D2-5 배치 축 열기
6	15:00–15:50	조용히 틀리는 export 3 가지 — if · NumPy · 고정 배치	강의 + 실습	D2-6 녹화되는 길
7	16:00–16:50	미지원 연산 고치기 · onnxslim 단순화	강의 + 실습	D2-7 미지원 · 단순화
8	17:00–17:50	미니 프로젝트 — 고장 난 BrightNet 고쳐 배포하기	개인 실습	D2-8 BrightNet

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 확인 (노트북) → 점검 (워크시트)

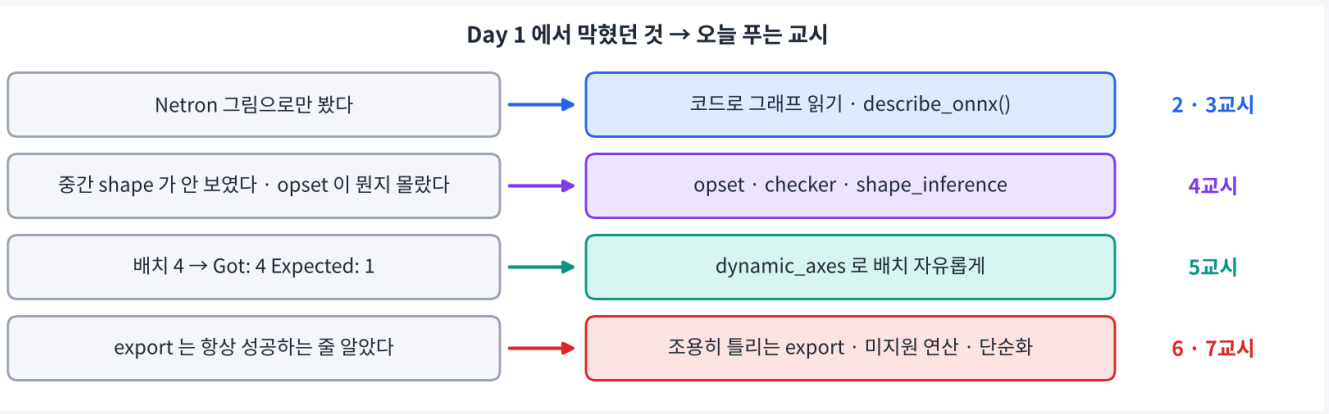
기본 주소 (Day 2)

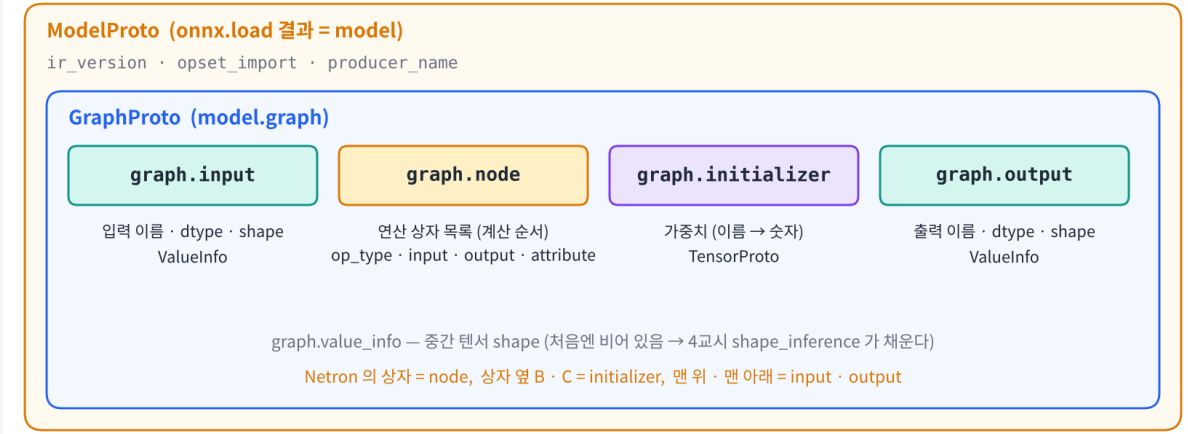
https://imguru-mooc.github.io/AI_CAMPUS/OT-day2/

animation.html	애니메이션 8 개 (D2-1 ~ D2-8) — #d2-1 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기 (교시당 10 문항)
OT_Day2_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 미니 프로젝트 (Colab 에서 열기)
OT_Day2.pdf	오늘 슬라이드 PDF
Day 1 복습 퀴즈	1 교시 09:00 — Google Forms 20 문항 (강사가 링크 공유)
data/	mnist_cnn.pt · mnist_sample.npz · 사진 3 장 (어제와 같은 파일)
pip install onnxslim	오늘 새로 쓰는 패키지 — 설치 셀에 포함

오늘의 큰 그림 — 어제 막혔던 4 가지를 푼다

내보내기 → 열어보기 → 실행하기 → 확인하기 중 ' 열어보기 ' 를 코드로





오늘 만나는 모델

SimpleConvNet — 1–5 교시

노드 7 · 가중치 텐서 6 · 433,890

GELU 모델 — 4 교시

opset 13 → 9 노드 · 20 → 2 노드

Gate · Center · Hard — 6 교시

조용히 틀리는 export 3 가지

MedianAlarm · ViewNet — 7 교시

aten::median · 11 → 4 노드

BrightNet — 8 교시

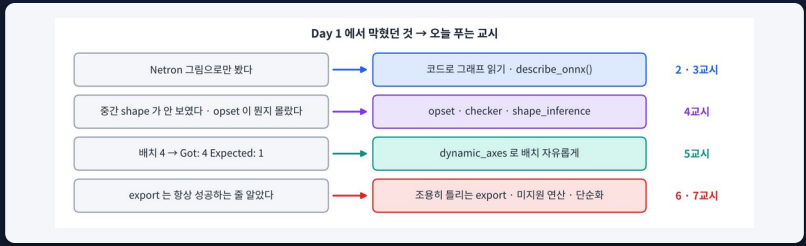
고장 3 곳 → 배포 파일 (opset 17)

1 Day 1 복습 · 4 단계 다시 · 오늘의 지도

09:00–09:50

학습 목표

- Day 1 복습 퀴즈 20 문항 (15 분)
- eval → export → 열어보기 → run · allclose
- mnist_cnn.onnx = 노드 7 · 가중치 텐서 6
- 어제 막힌 4 가지 → 오늘 교시



노트북 실습

1. 복습 퀴즈 (Google Forms)
2. 문제 1-1 4 단계 한 셀로
3. 문제 1-2 노드 · 가중치 수 예측

▶ 애니메이션 D2-1 4 단계 다시 · 오늘의 지도

핵심 개념

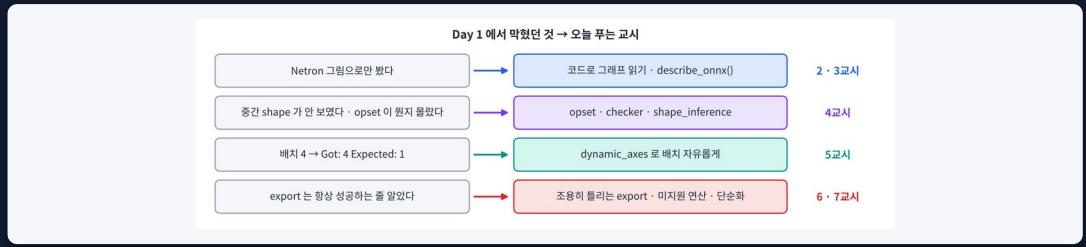
4 단계

```
cnn.eval()
torch.onnx.export(cnn, torch.zeros(1, 1, 28, 28), ...)
y = sess.run(None, {"input": x0})[0]
np.allclose(y, y_pt)
```

예측

```
노드 7 (Conv Relu MaxPool Flatten Gemm Relu Gemm)
가중치 텐서 6 (conv · fc1 · fc2 의 weight · bias)
```

오늘의 지도



체크포인트

어제 빠뜨린 단계 = 2. 열어보기. 오늘은 Netron 대신 코드로 연다.

2 ONNX 파일 = 그래프

10:00–10:50

학습 목표

- onnx.load → ModelProto → graph
- graph 의 4 목록 : input · node · initializer · output
  - 노드 = op_type + input 이름 + output 이름
  - 이름이 같으면 화살표



노트북 실습

1. 문제 2-1 네 목록의 길이
2. 문제 2-2 shape_of (dim_value · dim_param)
3. 문제 2-3 데이터 vs 가중치 입력
4. 문제 2-4 화살표 따라가기

▶ 애니메이션 D2-2 ONNX 파일의 속 구조

핵심 개념

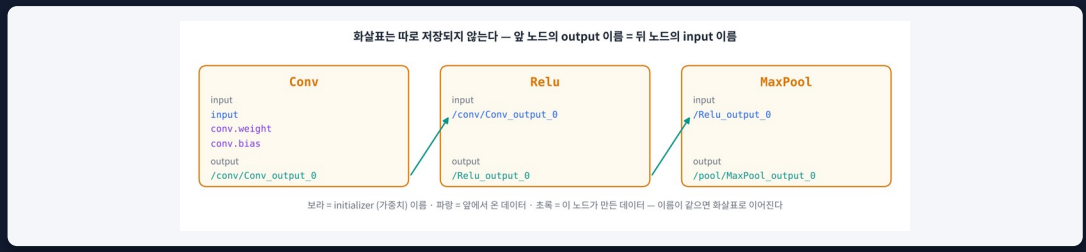
모델 정보

```
m = onnx.load("mnist_cnn.onnx"); g = m.graph
ir_version 9 · opset 20 · producer pytorch
```

네 목록

```
len(g.input), len(g.output), len(g.node), len(g.initializer)
# 1, 1, 7, 6  — 가중치는 input 이 아니다
```

화살표 = 이름



체크포인트

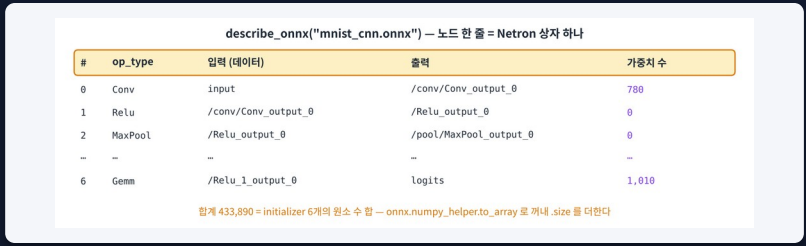
ONNX 는 노드를 계산 순서대로 저장한다 — 화살표를 따라간 순서 = graph.node 순서 .

3 가중치 · 속성 꺼내기 — describe_onnx()

11:00–11:50

학습 목표

- 가중치 = numpy_helper.to_array
- 속성 = helper.get_attribute_value
- 노드 한 줄 = Netron 상자 하나
- ONNX 가중치 수 ≠ PyTorch 파라미터 수 (BN)



노트북 실습

1. 문제 3-1 { 이름 : 배열 } · 합계 433,890
2. 문제 3-2 Conv · MaxPool 속성
3. 문제 3-3 describe_onnx 표
4. 🔁 SafetyNet 1491 vs 1443

▶ 애니메이션 D2-3 가중치 · 속성 꺼내기

핵심 개념

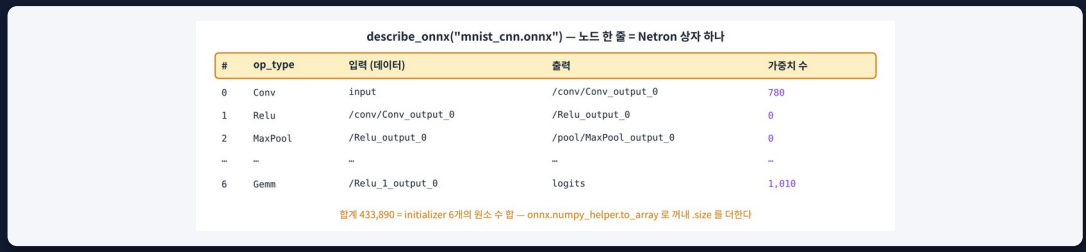
가중치

```
W = {t.name: numpy_helper.to_array(t)
     for t in g.initializer}
sum(a.size for a in W.values())   # 433,890
```

속성

```
Conv     kernel_shape [5, 5] · strides [1, 1]
MaxPool  kernel_shape [2, 2] · strides [2, 2]
Gemm     transB 1
```

요약 표



체크포인트

SafetyNet: PyTorch 1,491 vs ONNX 1,443 — 차이 48 = BN γ · β 가 Conv 에 합쳐짐 .

4 opset · checker · shape_inference

13:00–13:50

학습 목표

- opset = 연산 사전의 판 번호
  - 같은 GELU 도 판에 따라 9 노드 / 2 노드
- checker = 규칙 검사 (끊긴 화살표)
- shape_inference = 중간 shape 채우기



노트북 실습

1. 문제 4-1 GELU opset 13 vs 20
2. 문제 4-2 끊긴 화살표 고치기
3. 문제 4-3 value_info 0 → 6

▶ 애니메이션 D2-4 opset 사전 · checker

핵심 개념

opset

```
opset_version=13  →  Gemm Div Erf Add Mul … (9 개)
opset_version=20  →  Gemm, Gelu (2 개)
```

checker

```
ValidationError: Nodes in a graph must be
topologically sorted, however input 'nope'
of node /Relu is not output of any previous nodes.
```

shape



체크포인트

세 도구 모두 '결과가 맞는지' 는 말해 주지 않는다 → allclose 는 따로 .

5 배치 크기 자유롭게 — dynamic_axes

14:00–14:50

학습 목표

- { 이름 : { 축 번호 : 축 이름 }}
- shape 첫 축이 숫자 1 → 이름 'batch'
- 100 장을 run 한 번에
- dynamic 은 약속일 뿐 — 모델이 정한다



노트북 실습

1. 문제 5-1 dynamic_axes export
2. 문제 5-2 100 장 한 번에 (정확도 1.0)
3. 🔁 1 장씩 100 번 vs 100 장 1 번
4. 문제 5-3 H · W 를 열면 Gemm 실패

▶ 애니메이션 D2-5 dynamic_axes — 배치 축 열기

핵심 개념

export

```
dynamic_axes={"input": {0: "batch"},
              "logits": {0: "batch"}}
# ['batch', 1, 28, 28] → ['batch', 10]
```

100 장

```
X = (digits.astype(np.float32) / 255)[:, None]
# (100, 1, 28, 28) → 정확도 1.0
pred = sess_dyn.run(None, {"input": X})[0].argmax(1)
```

32×32

```
GEMM: Dimension mismatch, W: {100,4320} K: 5880
30 · 14 · 14 = 5880 ≠ 4320 → 배치 축만 연다
```

체크포인트

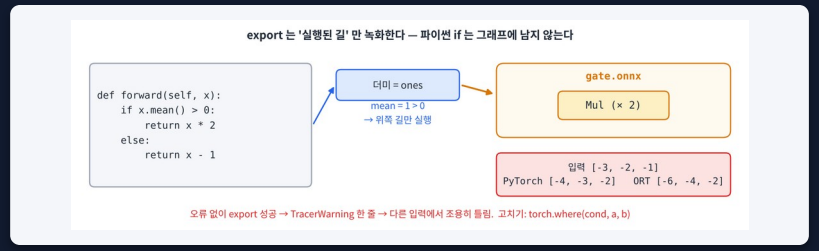
출력 쪽 축도 함께 적는 습관 . 축 이름은 input_names 와 같은 이름으로 .

6 조용히 틀리는 export 3 가지

15:00–15:50

학습 목표

- export = 실행된 길만 녹화
- 파이썬 if → 한쪽 길만 (torch.where)
- NumPy · float() → 상수로 굳음
- view(1, -1) → 배치 4 에서 오류



노트북 실습

1. verify() · export_warn() 도구
2. 문제 6-1 Gate → torch.where
3. 문제 6-2 Center → x.mean
4. 문제 6-3 Hard → flatten(1)

▶ 애니메이션 D2-6 녹화되는 길 · 굳는 값

핵심 개념

if

```
입력 [-3, -2, -1]
PyTorch [-4, -3, -2] · ORT [-6, -4, -2]
gate.onnx = [Constant, Mul]  ← else 없음
```

NumPy

```
m = float(np.mean(x.numpy()))  # Constant 5
고치기 : x - x.mean(dim=1, keepdim=True)
```

세 가지



체크포인트

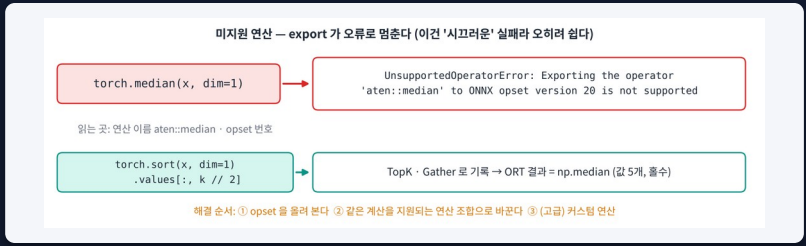
TracerWarning 한 줄이 유일한 신호 . verify 는 더미가 아닌 ' 다른 경우 ' 입력으로 .

7 미지원 연산 고치기 · onnxslim 단순화

16:00–16:50

학습 목표

- 오류에서 'aten:: 연산 ' · opset 읽기
- opset 올리기 또는 지원 연산으로 바꾸기
- 바꾼 뒤에는 원래 모델과 verify
- onnxslim = 상수 접기 · 노드 제거



노트북 실습

1. 문제 7-1 err_op = aten::median
2. 문제 7-2 torch.sort 로 중앙값
3. 문제 7-3 onnxslim 11 → 4
4. 🔁 접기 끔 16 → 4

▶ 애니메이션 D2-7 미지원 연산 · 단순화

핵심 개념

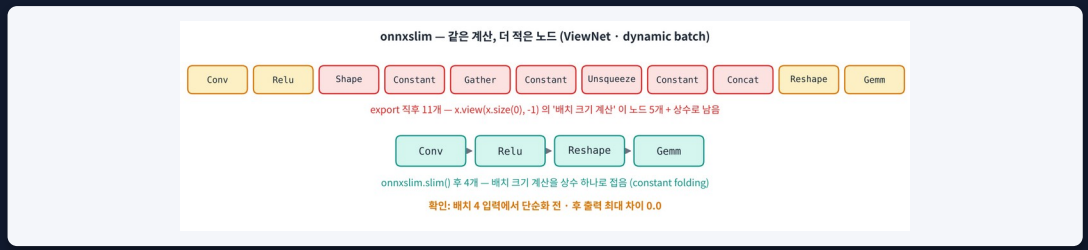
오류

```
UnsupportedOperatorError: Exporting the operator
'aten::median' to ONNX opset version 20 is not supported
```

바꾸기

```
med = torch.sort(x, dim=1).values[:, k:k + 1]
verify(med_net, "median_fix.onnx", cases_m)
#  원래 모델과 비교 → TopK  로 기록
```

단순화



체크포인트

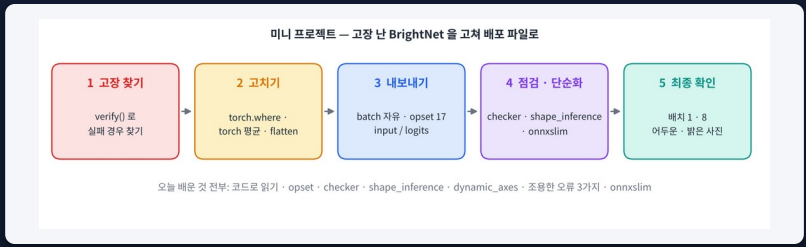
onnxslim.slim(path) 은 모델 객체를 돌려준다 → onnx.save. 단순화 후에도 allclose.

8 미니 프로젝트 — 고장 난 BrightNet

17:00–17:50

학습 목표

- ① if ② NumPy 평균 ③ view(1, -1)
- 사진마다 : dim=(1, 2, 3), keepdim=True
- batch · opset 17 · input / logits
- checker → onnxslim → shape_inference



노트북 실습

1. 과제 1 고장 찾기 (10)
2. 과제 2 고치기 (30)
3. 과제 3 배포용 export (20)
4. 과제 4 점검 · 단순화 (20)
5. 과제 5 · 6 최종 확인 · 해석 (20)

▶ 애니메이션 D2-8 BrightNet 고치기

핵심 개념

고장 찾기

더미 그대로 ✅ · 다른 밝은 사진 ❌ 3.9e-03
어두운 사진 ❌ 6.9e-03
→ 더미 (orange) 의 길 · 평균만 녹화됨

기준

intended(x) = 사진을 한 장씩 넣은 PyTorch 결과
배치 1 · 8 모두 최대 차이 < 1e-4

최종 파일

brightnet_final.onnx — 노드 16 → 14 · 중간 shape 27 · opset 17

체크포인트

채점 셀 자동 90 점 + 과제 6 서술 10 점 . 채점 셀 결과가 보이는 상태로 공유해 제출 .

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

파일 구조	onnx.load → graph 의 input · node · initializer · output — 가중치는 input 이 아니다	D2-2
화살표	앞 노드의 output 이름 = 뒤 노드의 input 이름 · 노드는 계산 순서대로	D2-2
describe_onnx	to_array · get_attribute_value 로 노드마다 가중치 수 · 속성 표	D2-3
opset	연산 사전 판 번호 · GELU 13 → 9 노드 / 20 → 2 노드 · opset_version= 으로 정한다	D2-4
checker · shape	checker = 규칙 검사 · shape_inference = 중간 shape · 결과 검사는 allclose	D2-4
dynamic_axes	{"input": {0: "batch"}} · 100 장 한 번에 · 모델이 못 받는 크기는 실행 중 오류	D2-5
조용한 오류	if → torch.where · NumPy → torch 연산 · view(1, -1) → flatten(1) · 여러 입력으로 verify	D2-6
미지원 · 단순화	aten:: 오류 → 지원 연산으로 · onnxslim 11 → 4 · 전후 비교	D2-7

Day 3 로 가는 다리 — ONNX Runtime 으로 빠르게

오늘 '느낌만' 본 속도를 제대로 잰다

Day 2 (오늘)		Day 3 (11/6 금)
1 장씩 100 번 vs 100 장 1 번 (느낌)	→	bench() — warm-up · 평균 · p95 · FPS
CPUExecutionProvider 만	→	CUDAExecutionProvider — 한 단어로 GPU
onnxslim 으로 노드 줄이기	→	ORT 그래프 최적화 레벨 · 노드 합치기 보기
mnist_cnn_dyn.onnx (배치 자유)	→	배치 1 · 4 · 16 · 64 처리량 곡선
describe_onnx · verify	→	그대로 사용 — 성능을 바꿀 때마다 결과 확인
BrightNet 최종 파일	→	profiling — 어느 노드가 오래 걸리나

내보내기 → 열어보기 → 실행하기 → 확인하기 — 내일은 '실행하기' 를 빠르게